How Many Servers Do You Need? Capacity, 60% Target & N+2

measured capacity, a safe target, and room for failures

Watch the 4-min video

In one line: servers = peak RPS ÷ (measured per-instance capacity × a 60–70% utilization target), then add redundancy (N+1 or N+2) and spread across availability zones; and check that the databases and caches behind them can take the same load.

1 Three numbers, not one

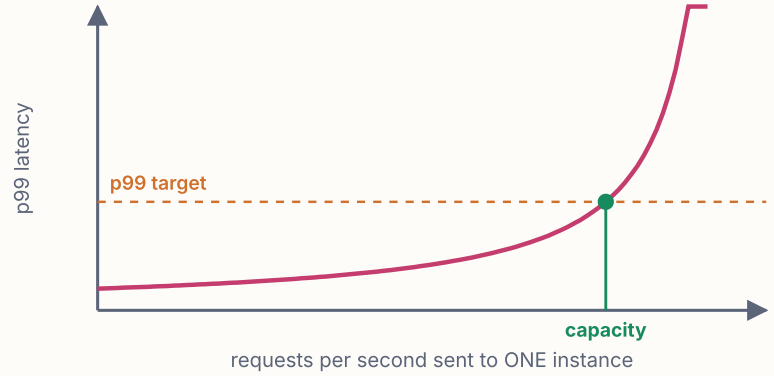
Your API must handle **12,000 requests per second at peak**. How many servers, or Kubernetes pods, do you need? Dividing by a number you heard somewhere is not enough. You need three things:

1. the **capacity of one instance**, measured;
2. a **safe utilization target**;
3. **room for failures**.

```
instances = peak RPS ÷ (capacity per instance × target) + redundancy
```

2 Measure per-instance capacity

Run a **load test** against one instance and increase traffic until latency, such as the **99th percentile (p99)**, crosses your target. That throughput is the instance's capacity. Rules of thumb vary hugely: a simple cached endpoint might handle thousands of requests per second per core, a heavy endpoint only tens.



Measure, don't guess

Ramp traffic on one instance until p99 crosses the target. That RPS is its capacity. Cached endpoint: thousands of RPS per core. Heavy one: tens.

Figure 1. Capacity is the highest RPS at which p99 still meets the target, not the point where the server falls over.

3 CPU-bound or I/O-bound?

	CPU-bound	I/O-bound
Examples	image resizing, encryption, model inference	calling databases and other services
Where time goes	computing	waiting on the network or disk
Scales with	cores: more requests need more CPUs	concurrency: async I/O lets one instance hold many requests
Real limit	CPU	often the database or the connection pool

For I/O-bound services, Little's Law helps: an instance doing 1,000 RPS at 50 ms per request holds about 1,000 × 0.05 = 50 requests in flight, so it needs a pool and connections for at least 50.

4 Never plan to run at 100%

Latency rises sharply as utilization nears the limit, because requests queue up. A common target is **60–70% at peak**. That leaves headroom for spikes, garbage-collection pauses, noisy neighbours and uneven load balancing.

WORKED EXAMPLE: 12,000 RPS

Each pod handles **1,000 RPS** at its limit. At a **60% target**, each pod should take 600. 12,000 ÷ 600 = **20 pods**.

5 Add redundancy and zones

- **N+1**: you can lose any one instance and still handle peak load.
- **N+2**: you survive a failure *during* a deployment or maintenance, when one instance is already out.
- **Availability zones**: spread instances across at least **3 zones**. If one zone fails, the other two must carry the whole load, which can mean **50% more instances**.

So 20 pods becomes about **22 with N+2**, or **30 for full zone tolerance** (10 per zone, so any two zones still have the 20 you need).

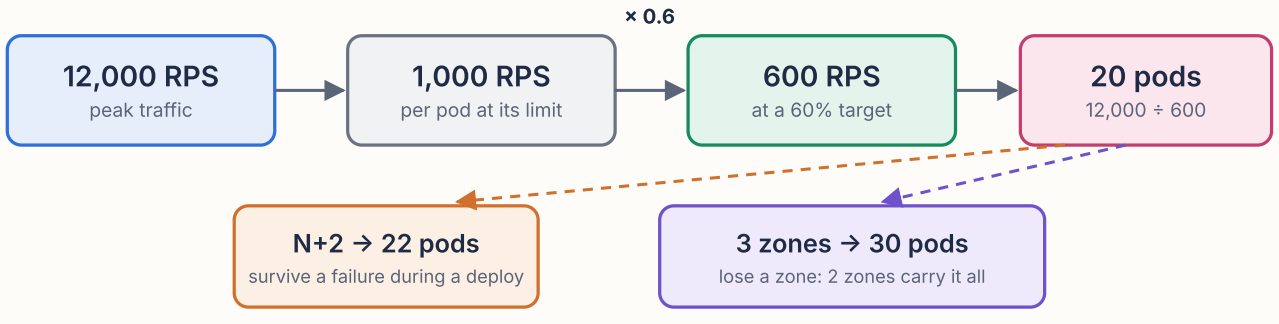


Figure 2. From peak traffic to a pod count, then two ways to add safety: N+2 for instance failures, and 50% extra for losing a whole zone.

6 Interview question: 20,000 RPS

How many servers for 20,000 requests per second? Say one server handles **2,000 RPS** at its limit, and we target 60%, so **1,200** each. 20,000 ÷ 1,200 ≈ 16.7, round up to **17**. Add N+2: **19**. Spread across three

zones, rounding up to 7 per zone: **21 servers**.

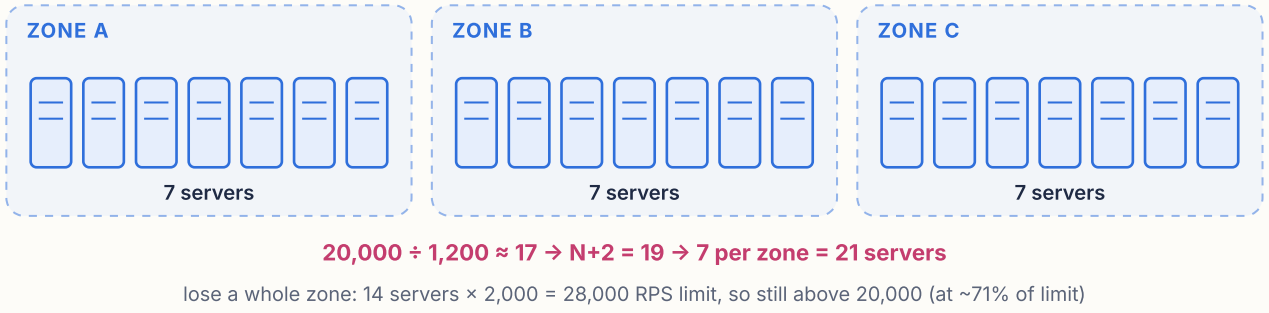


Figure 3. 21 servers over three zones. Losing a zone leaves 14, which still stay under their limit, though above the 60% target until autoscaling adds more.

Remember the dependencies. The database and caches behind the servers must handle the same load. 20,000 RPS with two queries per request is 40,000 queries per second on the data tier, unless the cache absorbs most of them.

COMMON MISTAKES

- Using a capacity number from a blog instead of a load test.
- Planning to run at 90–100% utilization.
- Forgetting redundancy, so one failure or one deploy means overload.
- Putting all instances in one zone.
- Scaling the app tier and forgetting the database and connection pool behind it.

7 Key terms

Per-instance capacity: max RPS while p99 meets target

Load test: ramp traffic to find capacity

Utilization target: planned busy share, 60–70%

CPU-bound: limited by computation

I/O-bound: limited by waiting on I/O

N+1 / N+2: spare instances for 1 / 2 failures

Availability zone: isolated data center in a region

Headroom: spare capacity for spikes

p99: 99% of requests are faster

8 Interview questions

Q1. How many servers do you need for 20,000 RPS? BASIC

Measure capacity with a load test (say 2,000 RPS), apply a 60% target (1,200), divide: about 17. Add N+2 (19) and spread across three zones: about 21.

Q2. Why target 60–70% utilization instead of 100%? BASIC

Latency rises sharply near the limit because requests queue. Headroom covers spikes, GC pauses, noisy neighbours and uneven load balancing.

Q3. How do you find one instance's capacity? BASIC

Load-test a single instance, ramping traffic until p99 latency crosses the target. The RPS at that point is its capacity.

Q4. 12,000 RPS peak, 1,000 RPS per pod at its limit. How many pods? SCENARIO

At 60%, 600 per pod: 12,000 ÷ 600 = 20. N+2 gives 22; full zone tolerance across 3 zones gives 30.

Q5. Your service mostly calls a database. What limits it, and how do you scale it? SCENARIO

It is I/O-bound: the limit is often the database or connection pool, not CPU. Use async I/O, size pools with Little's Law, and make sure the database can take the total load before adding app servers.

Q6. Why can surviving a zone failure need 50% more instances? TRICKY

With 3 zones, losing one leaves two-thirds of the fleet. To still serve full load, each zone must hold half of what you need: 20 needed → 10 per zone → 30 total.

Q7. You added servers but latency did not improve. Why? TRICKY

The bottleneck is elsewhere: the database, a cache, a connection pool or a downstream service. Find the saturated resource before scaling the app tier.

Next: SD-D-010 · Growth Projections